spark / GAME DESIGNER

ภารกิจ 02

สร้างผู้เล่นที่เคลื่อนไหว

เริ่มไฟล์เดียว ให้ผู้เล่นมีสถานะของตน แล้วแยกตัวช่วยฉากเพื่อใช้ซ้ำ

01 ลองทาย 02 ตามรอยโค้ด 03 อธิบาย

PLAY · THINK · CODE · CREATE · GROW

ภาพจินตนาการ / สำรวจว่าโค้ดทำอะไร



ไฟล์เดียว เห็นลูปเกมครบ

import ทำให้มีเครื่องมือ ลูปของเรากำหนดว่าจะใช้เมื่อไร

อ่านโค้ดด้วยกัน

```
import pygame
while running:
    keys = pygame.key.get_pressed()
    pygame.display.flip()
```

01
นำเครื่องมือมาใช้
import pygame

→

02
อ่านและอัปเดต
keys x, y



→

03
แสดงเฟรม
display.flip()



ลองทายดู: import pygame ทำให้หุ่นเดินไหม?

มาดูคำตอบกัน: ไม่ ลูปอ่านปุ่ม อัปเดตตำแหน่ง และวาดทุกเฟรม

อ้างอิง: 01_loop.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

ขยับทีละก้าว แล้วอีกก้าว

ลูปจำกัด dt ที 0.05 วินาที speed เป็นพิกเซลต่อวินาที

อ่านโค้ดด้วยกัน

```
dt = min(clock.tick(60) / 1000, 0.05)
x += dx * speed * dt
y += dy * speed * dt
```

เริ่ม

x = 160

ขวา: อัปเดตครั้งหนึ่ง

1 × 180 × 0.05

x = 169

ขวา: อัปเดตถัดไป

x = 178

ลองทายดู: ปล่อยปุ่ม dx=0 ครั้งถัดไปเปลี่ยนอะไร?

มาดูคำตอบกัน: ตำแหน่งคง 178 แต่ลูปยังวาดต่อ

อ้างอิง: 01_loop.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

ตังชือให้งานวาดของเรา

helper คือฟังก์ชันสำหรับงานหนึงอย่าง เริมเก็บใน main.py ก่อน

อ่านโค้ดด้วยกัน

```
def draw_scene(screen, pad, pad_position):
    screen.fill("#DDF2FF")
    pygame.draw.rect(screen, "#95C549", (0, 420, 800, 180))
    pygame.draw.rect(screen, "#F7D34B", (80, 390, 640, 18))
    screen.blit(pad, pad.get_rect(center=pad_position))
```

01
ล้าง
sky colour

→

02
สร้างฉาก
ground + route + landing pad



→

03
วาดผู้เล่นท้ายสุด
robot on top



ลองทายดู: ถ้าล้างจอหลังวาดผู้เล่นจะเกิดอะไร?

มาดูคำตอบกัน: ภาพผู้เล่นถูกลบ ล้างก่อน วาดพืนหลัง แท่น แล้วผู้เล่น

รู้จักแบบของ Player

คลาสอธิบายผู้เล่น เรียก Player(...) เพื่อสร้าง instance

อ่านโค้ดด้วยกัน

```
class Player:
    def __init__(self, x, y, image):
        self.x = x
        self.y = y
        self.speed = 180
        self.image = image
player = Player(160, 300, robot_image)
```

01

คลาส

Player

state + methods

02

instance

player

x = 160

y = 300

speed = 180



ลองทายดู: รูปภาพคือออบเจ็กต์ผู้เล่นทั้งหมดไหม?

มาดูคำตอบกัน: ไม่ instance ยังมีตำแหน่งและความเร็ว และใช้เมธอด move/draw

อ้างอิง: main.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

self คือผู้เล่นตัวนี้

เมธอดคือฟังก์ชันของออบเจ็กต์ การเรียกเลือกผู้รับคำสั่ง

อ่านโค้ดด้วยกัน

```
def move(self, dx, dy, dt):
    self.x += dx * self.speed * dt
    self.y += dy * self.speed * dt
    self.x = max(32, min(768, self.x))
    self.y = max(32, min(568, self.y))
player.move(dx, dy, dt)
```

01 ก่อน

player.x = 160



02 เรียกเมธอด

dx=1, dy=0
dt=0.05
self = player

03 หลัง

player.x = 169



ลองทายดู: player.draw(screen) เปลี่ยน x ไหม?

มาดูคำตอบกัน: ไม่ move เปลี่ยนสถานะ draw อ่านสถานะเพื่อวางภาพ

อ้างอิง: main.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

สองผู้เล่น สองตำแหน่ง

ตัวอย่างนี้ไล่เฉพาะตำแหน่ง ใช้ None เพราะไม่วาดภาพ

อ่านโค้ดด้วยกัน

```
nova = Player(160, 300, None)
pip = Player(500, 300, None)
nova.move(1, 0, 0.05)
print(nova.x, pip.x)
```

01
ก่อน

nova.x = 160
pip.x = 500

02
ขยับแค่ Nova

nova.x = 169



03
Pip อยู่ทีเดิม

pip.x = 500



ลองทายดู: สร้าง Pip แล้ว Nova จะเริ่มใหม่ไหม?

มาดูคำตอบกัน: ไม่ แต่ละ instance มีแอตทริบิวต์ของตน self เป็น Nova เฉพาะการเรียก nova.move

อ้างอิง: object_probe.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

เส้นทางเดิม ความเร็วใหม่

คงจุดเริ่มและเป้าหมายเพื่อเทียบอย่างยุติธรรม

อ่านโค้ดด้วยกัน

```
self.speed = 240
self.x += dx * self.speed * dt
```

01

เดิม

180 × 0.05

9 pixels



02

ปรับแล้ว

240 × 0.05

12 pixels



ลองทายดู: ความเร็วใดช่วยหยุดบนแท่น? ทดลองสิ่งที่คาด

มาดูคำตอบกัน: ความชอบต่างกันได้ ใช้ผลลองจอดอธิบายข้อแลกเปลี่ยน

อ้างอิง: teacher_example.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

ย้าย helper ไปใช้ซ้ำ

ย้ายนิยาม helper สองตัวไป scene.py คงลูปและ Player ใน main.py

อ่านโค้ดด้วยกัน

```
from scene import load_sprite, draw_scene
robot_image = load_sprite(images / "robot.png", 64)
draw_scene(screen, pad, pad_position)
player.draw(screen)
```

01
นิยาม
scene.py
load_sprite
draw_scene

→

02
นำเข้า
main.py
from scene
import ...

→

03
เรียก
once: load_sprite
each frame:
draw_scene

ลองทายดู: import draw_scene แล้ววาดเฟรมเลยไหม?

มาดูคำตอบกัน: ไม่ import ทำให้ใช้ helper ได้ การเรียกในลูปจึงวาดฉาก

อ้างอิง: modular/main.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

ห้อง sprite · หันได้สี่ทิศ

หุ่นหนึ่งตัว สี่ภาพ กล้องอยู่เหนือพื้น ภาพเปลี่ยนทิศยังไม่ใช่แอนิเมชันเดิน

อ่านโค้ดด้วยกัน

```
    "up": load_sprite(images / "robot_up.png", 64),
    "right": load_sprite(images / "robot_right.png", 64),
    "down": load_sprite(images / "robot_down.png",
64),
    "left": load_sprite(images / "robot_left.png", 64),
```

01 ขึ้น
up



02 ขวา
right



03 ลง
down



04 ซ้าย
left



ลองทายดู: ภาพใดเห็นกระเป๋าด้านหลังชัดที่สุด?

มาดูคำตอบกัน: ขึ้น: เห็นด้านหลัง sprite คือภาพในเกม Player เก็บตำแหน่งและพฤติกรรม

ตู้เก็บภาพของเรา

dictionary จับคู่คีย์ทิศกับภาพ โหลดครั้งเดียวก่อนลูป

อ่านโค้ดด้วยกัน

```
self.sprites = sprites
self.facing = "down"
image = self.sprites[self.facing]
```

01 คีย์

"down"

→

02 ค่าภาพ

sprites["down"]



→

03 ภาพที่เลือก

image



ลองทายดู: ข้อความ "down" เป็นภาพหรือไม่?

มาดูคำตอบกัน: ไม่ใช่ ข้อความคือคีย์ การเลือกค่าจาก dictionary จะได้ภาพที่โหลดไว้

อ้างอิง: 04_sprites.py / ภาพอธิบาย / บรรทัดที่เลือกไม่ได้รันแยก

อินพุตเลือกทิศ

วางชุดเงื่อนไขนี้ใน Player.move แนวตั้งชนะเมื่อเดินเฉียง

อ่านโค้ดด้วยกัน

```
if dy < 0:
    self.facing = "up"
elif dy > 0:
    self.facing = "down"
elif dx < 0:
    self.facing = "left"
elif dx > 0:
    self.facing = "right"
```

01
ขึ้น + ขวา
dx=1, dy=-1

→

02
ทีงแรกทีจริง
dy < 0

→

03
หันขึ้น
facing="up"



ลองทายดู: ถ้าเงื่อนไข dx มาก่อนจะเปลียนอะไร?

มาดูคำตอบกัน: ภาพจะหันขวา ตำแหน่งยังเดินเฉียงเพราะยังอัปเดต x/y ทังคู่

อ้างอิง: 04_sprites.py / ภาพอธิบาย / บรรทัดทีเลือกไม่ได้รันแยก

หยุดเดิน แต่ยังหันเหมือนเดิม!

เมืออินพุตทั้งสองเป็นศูนย์ ไม่มีกิงใดเปลียน facing

อ่านโค้ดด้วยกัน

```
        self.x += dx * self.speed * dt
        self.y += dy * self.speed * dt
        image = self.sprites[self.facing]
        rectangle = image.get_rect(center=(round(self.x),
round(self.y)))
        screen.blit(image, rectangle)
```

01
กดขวา
right



02
ปล่อย
right



03
กดขึน
up



ลองทายดู: ถ้ามี else ตัง facing="down" จะจำทิศได้ไหม?

มาดูคำตอบกัน: ไม่ได้ จะหันลงเมือปล่อย การเลือกภาพและการเปลียนตำแหน่งเป็นคนละงาน

อ้างอิง: 04_sprites.py / ภาพอธิบาย / บรรทัดทีเลือกไม่ได้รันแยก

สองผู้เล่น สองทิศ

แต่ละออบเจ็กต์เก็บ facing ของตน ใช้คลังภาพเดียวกันได้

อ่านโค้ดด้วยกัน

```
def __init__(self, x, y, sprites):
    self.x = x
    self.y = y
    self.speed = 180
    self.sprites = sprites
    self.facing = "down"
```

01

เฉพาะ a เดินซ้าย

a: left



02

แล้ว a หยุด

a: left



03

b ไม่ได้เคลื่อนที

b: down



ลองทายดู: แม้ใช้ dictionary sprites เดียวกัน แอตทริบิวต์ใดต่างกัน?

มาดูคำตอบกัน: self.facing เปลียน a.facing ไม่เปลียน b.facing ลองไล่อินพุตใหม่ให้แต่ละตัว

อ้างอิง: 04_sprites.py / ภาพอธิบาย / บรรทัดทีเลือกไม่ได้รันแยก